

ENCORE

Ticket Booking Platform
System Design Write-Up

AJITESH SHARMA
VIT Vellore · Batch 2023 · Passout 2027
Computer Science & Engineering — Specialisation in Information Security

1. Architecture Overview

Encore is a three-tier, full-stack web application designed for high-concurrency ticket sales. Each tier is independently deployed and communicates over standard HTTPS. The **frontend** is a Next.js 15 (React) app hosted on **Vercel's global CDN**, ensuring fast page loads worldwide. The **backend API** is a NestJS (Node.js) server on **Render**, responsible for all business logic — authentication, seat management, bookings, payments, and background jobs. The **database** is a **Neon PostgreSQL** serverless instance that stores all persistent state including seats, bookings, holds, payments, and the waitlist queue.

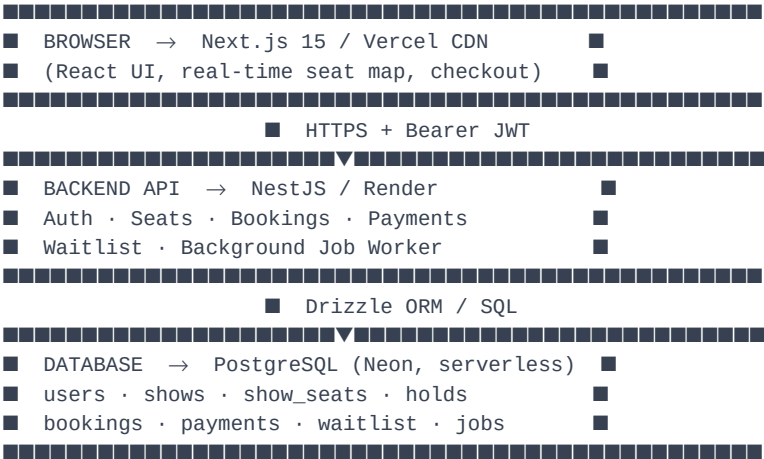


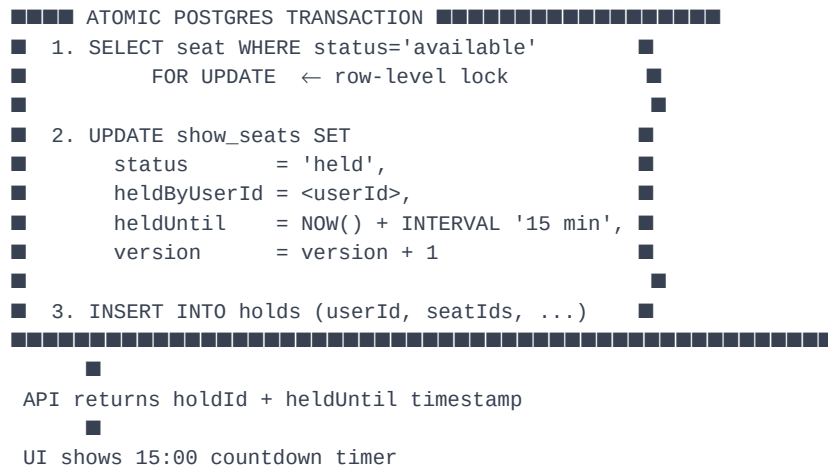
Figure 1 — Three-tier deployment architecture.

2. Seat Hold & TTL (Time-To-Live) Mechanism

When a user selects seats and proceeds to checkout, those seats cannot be sold to anyone else while the user is still paying — but they also cannot be locked forever if the user simply closes their browser. Encore solves this with a **15-minute TTL hold**.

The moment a user clicks 'Hold Seats', the API runs a single atomic database transaction that changes the seat's status from *available* to *held*, stamps the seat with the user's ID, and sets **heldUntil = NOW() + 15 minutes**. This timestamp is the TTL. Every subsequent API call — generating a payment intent, applying a discount, confirming the booking — cross-checks this timestamp server-side. If the clock has already passed **heldUntil**, the API responds with **HTTP 409 Conflict** immediately, making the cart invalid without waiting for any background process.

```
User selects seats
■
POST /api/shows/:showId/hold
■
```



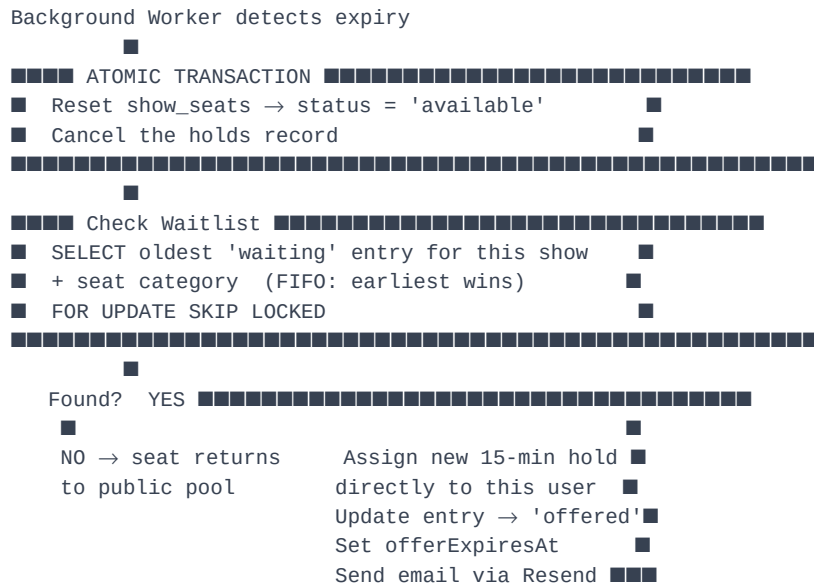


Figure 3 — Waitlist auto-allocation triggered by hold expiry.

5. Time-Limited Offer Handling

Once a waitlisted user receives their seat offer, they have exactly **15 minutes** to complete checkout — the same TTL as a regular hold. This is not just a courtesy window; it is enforced by the same server-side timestamp check. If the user ignores the email and the offer expires, the background worker detects the expired `offerExpiresAt` timestamp on the next cycle, marks the waitlist entry as *expired*, releases the hold, and immediately runs the allocation again for the next person in the queue. This creates a **self-healing cascade** — tickets continuously flow down the waitlist with zero manual organiser intervention, maximising revenue and venue capacity.

■ Background Worker runs every 30 seconds and handles all of this automatically:

- Finds all seats where `heldUntil ≤ NOW()` and resets them to *available*.
- Finds all waitlist offers where `offerExpiresAt ≤ NOW()` and marks them *expired*.
- For each newly freed seat, runs waitlist allocation to find the next FIFO candidate.
- Re-schedules itself by inserting a new job 30 seconds in the future (no cron daemon required).

6. Security Design

Security is built into every layer. The system uses industry-standard, battle-tested techniques — no custom crypto, no shortcuts.

Mechanism	Implementation	Why It Matters
JWT Auth	HS256, 7-day expiry, refresh-token rotation	Stateless — no session DB needed
Password Hashing	Argon2id (memory-hard)	Resistant to GPU brute-force
CORS Policy	Whitelist: Vercel + custom domain only	Blocks unauthorised cross-origin calls
Rate Limiting	NestJS Throttler: 100 req / 60s per IP	Prevents brute-force login attacks
SQL Safety	Drizzle ORM parameterised queries only	Zero SQL injection risk

Helmet	HTTP security headers on every response	Prevents XSS, clickjacking, MIME sniff
---------------	---	--